

CSC/CPE 315
Computer Architecture
Prof. Seng

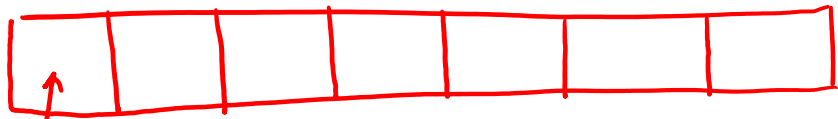
Number Representation and Intro. to MIPS Instruction Set

3.2 & 2.1-2.6

Number Representation

- Chapter 3.2
- Why do we use binary rather than decimal?
 - *because of circuit technology*
- Why do we use two's-complement for negative numbers?
 - *sign handling is automatic*

Two's complement


sign
bit
0 = positive
1 = negative

Negate:

1. flip the bits
2. add 1

+5 0 1 0 1
 ↓ ↓ ↓ ↓
step 1 1 0 1 0
step 2 + 1

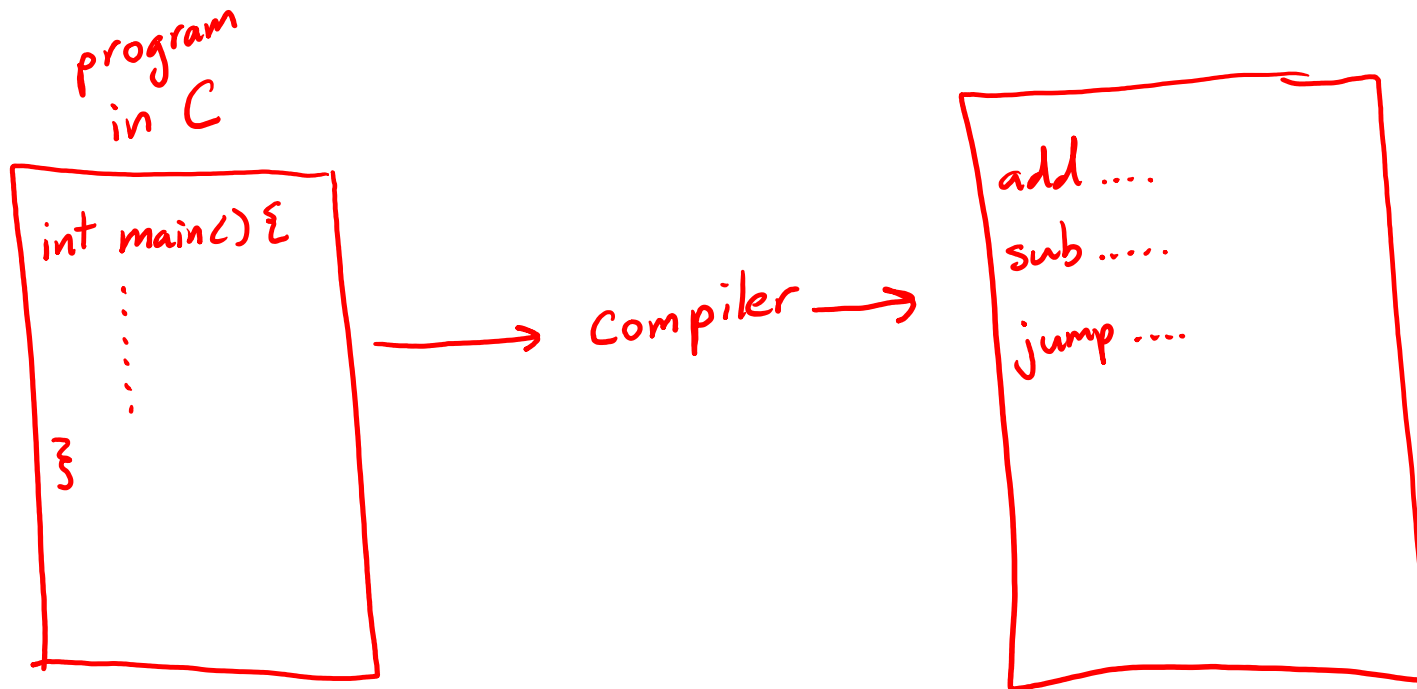
-5 1 0 1 1
 0 1 0 0
 + 1

+5 0 1 0 1

(A red arrow curves from the final result '0 1 0 1' back to the original '+5' and its bit representation '0 1 0 1').

Instruction Set Architecture (ISA)

- Definition: *Instruction-level programmer interface to a CPU*



HW/SW facts

- ^(CPU) Processor = fast (5 GHz = 5000 MHz)
- ^(RAM) Memory = slow (2-3 GHz)

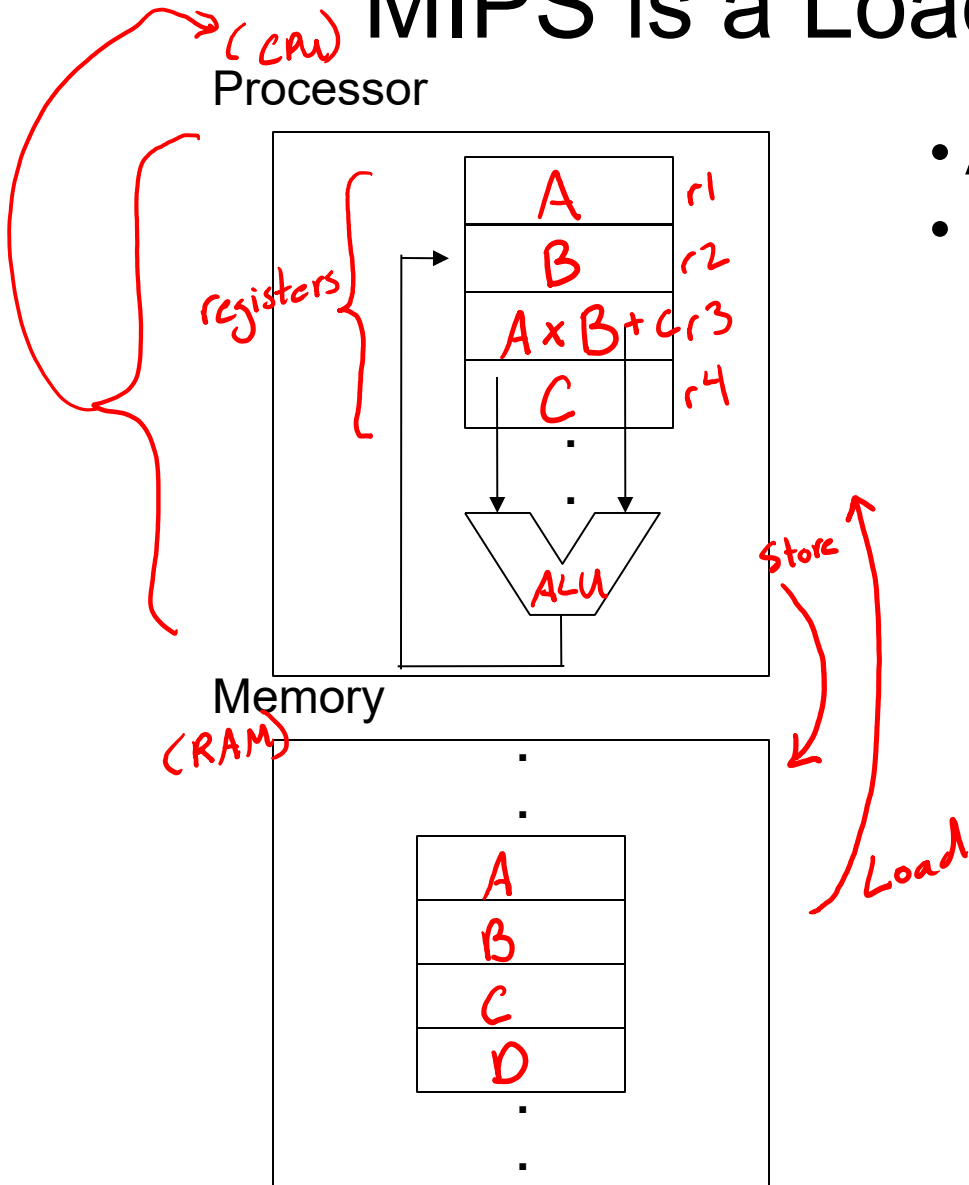
MIPS

- 1. • Load-Store architecture
 - Arithmetic operations only on registers
- 2. • Simple instructions (RISC)
 - Every instruction is a fixed length
- 32 total registers

Reduced
Inst.
Set
Computing



MIPS is a Load-Store Architecture



✓ Arithmetic Logic Unit

- ALU inputs: Two registers
- Memory access:

Load: copy a value from RAM into a register

Store: copy a value from a register into RAM

$$D = A * B + C$$

Load r1, A
Load r2, B
Mult r3, r1, r2
Load r4, C
Add r3, r3, r4
Store D, r3

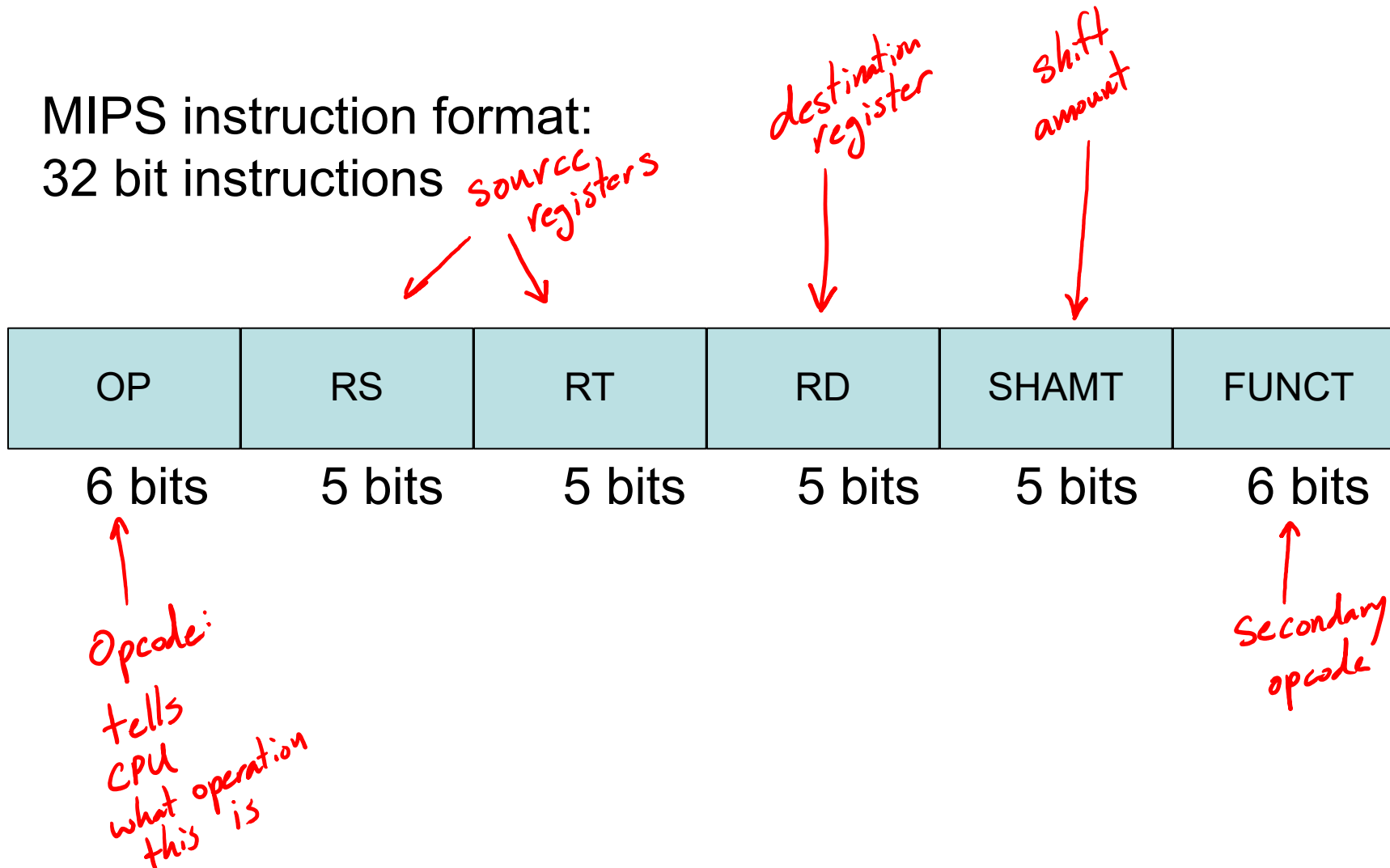
Disadvantage
of L/S:
— more inst.
required

Reasons for Load/Store

- Registers are fast – Memory is slow

MIPS Instructions

MIPS instruction format:
32 bit instructions



MIPS Registers – 32 registers

Name	Reg Number	Usage	Preserved across call?
<i>always</i> \$zero , \$0	0	The constant 0	Yes
\$v0-\$v1	2-3	Function results	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved	Yes
\$t8-\$t9	24-25	More temporaries	No
\$gp	28	Global pointer	Yes
\$sp	29	Stack pointer	Yes
\$fp	30	Frame pointer	Yes
\$ra	31	Return address	Yes

1. R-format Instructions

- Arithmetic instructions
 - Two input registers
 - One output register

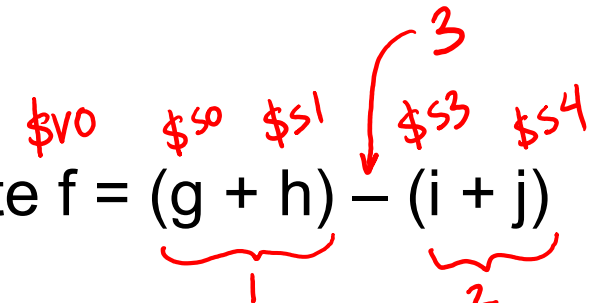
slt = set less than

slt \$s0, \$t0, \$t0
↑ gets 0

Operation	rs	rt	rd	shamt	funct	meaning
add \$2,\$3,\$5	3	5	2	0	32	\$2 = \$3 + \$5
sub \$2,\$3,\$5	3	5	2	0	34	\$2 = \$3 - \$5
addu \$2,\$3,\$5	3	5	2	0	33	\$2 = \$3 + \$5
slt \$2, \$3, \$5	3	5	2	0	42	if (\$3 < \$5) \$2 = 1 else \$2 = 0

MIPS Example 1

Translate $f = (g + h) - (i + j)$ into MIPS assembly



Assumption: g has been loaded into \$s0, h in \$s1, i in \$s3, and j in \$s4.

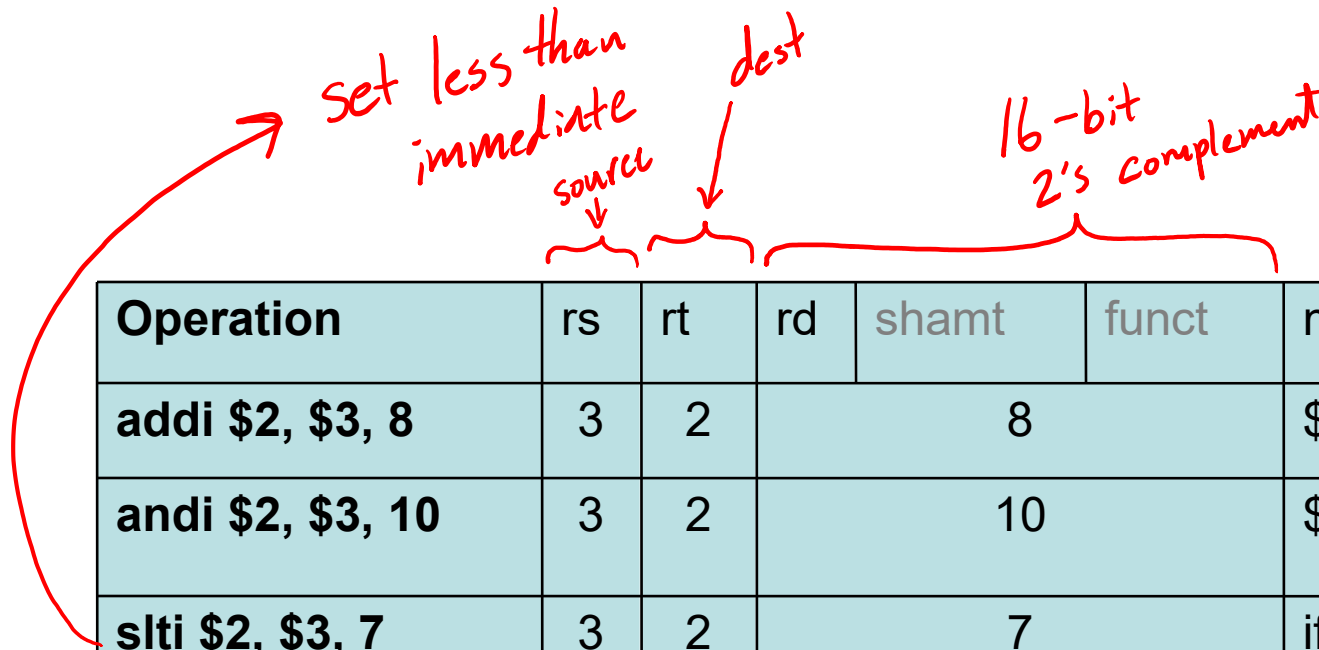
We want the result placed in \$v0.

1. `add $s0, $s0, $s1`
2. `add $s3, $s3, $s4`
3. `sub $v0, $s0, $s3`

2. I-format Instructions

immediate

- Operations involving register and constant



Operation	rs	rt	rd	shamt	funct	meaning
addi \$2, \$3, 8	3	2		8		\$2 = \$3 + 8
andi \$2, \$3, 10	3	2		10		\$2 = \$3 & 10
slti \$2, \$3, 7	3	2		7		if (\$3 < 7) then \$2=1 else \$2=0